



SETUP GUIDE

Architecture Decision Note (ADN) Template

Tools: ONNX Runtime (required), TensorRT (optional, NVIDIA GPU),

OpenVINO (optional, Intel CPU)

Prerequisite: Python 3.9+ with pip, Docker (from Week 4)

SECTION 1: WSL (Windows Subsystem for Linux) — RECOMMENDED FOR WINDOWS

PREREQUISITES

- WSL2 with Ubuntu (from Week 4)
- Python 3.9+ (python3 --version)
- pip installed (pip3 --version)
- (Optional) NVIDIA GPU with CUDA for TensorRT/GPU acceleration

STEP 1: Create a virtual environment

```
cd ~
```

```
python3 -m venv opt-env
```

```
source opt-env/bin/activate
```

STEP 2: Install ONNX Runtime (CPU — works for everyone)

```
pip install onnx onnxruntime numpy onnxscript
```

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

NOTE: These must be two separate commands. The --index-url flag replaces PyPI entirely, so onnx/onnxruntime/numpy/onnxscript would fail if combined.

NOTE: onnxscript is required by newer versions of PyTorch for ONNX export (torch.onnx.export). Without it you will get "No module named 'onnxscript'".

For GPU support (requires CUDA 11.x or 12.x installed):

```
pip install onnxruntime-gpu
```

Verify:

```
python -c "import onnxruntime as ort; print('ONNX Runtime:', ort.__version__); print('Device:', ort.get_device())"
```

Expected: ONNX Runtime: 1.XX.X, Device: CPU (or GPU)

STEP 3: Install ONNX model tools

```
pip install onnx onnxconverter-common
```

Verify:

```
python -c "import onnx; print('ONNX:', onnx.__version__)"
```

STEP 4: (Optional) Install TensorRT — requires NVIDIA GPU + CUDA

```
# Verify CUDA is available
```

```
nvidia-smi
```

```
nvcc --version
```

```
# Install TensorRT via pip (simplest method)
```

```
pip install tensorrt
```

```
# Verify
```

```
python -c "import tensorrt as trt; print('TensorRT:', trt.__version__)"
```

Note: TensorRT installation can be complex. If pip install fails:

- Ensure CUDA toolkit matches TensorRT requirements
- Consider using NVIDIA's NGC Docker containers (pre-configured)
- See NVIDIA docs: <https://docs.nvidia.com/deeplearning/tensorrt/install-guide/>

STEP 5: (Optional) Install OpenVINO — best on Intel CPU

```
pip install openvino openvino-dev
```

Verify:

```
python -c "import openvino as ov; print('OpenVINO:', ov.__version__)"
```

STEP 6: Run Complete Verification Test

```
python3 << 'EOF'
import torch
import torch.nn as nn
import onnxruntime as ort
import numpy as np
import time
```

1. Create a simple model

```
class SimpleModel(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
```

```
self.relu = nn.ReLU()
self.fc2 = nn.Linear(256, 10)
def forward(self, x):
    return self.fc2(self.relu(self.fc1(x)))
model = SimpleModel()
model.eval()
```

2. Export to ONNX

```
dummy_input = torch.randn(1, 784)
torch.onnx.export(model, dummy_input, "/tmp/test_model.onnx",
    input_names=['input'], output_names=['output'])

print("✓ Model exported to ONNX")
```

3. Load in ONNX Runtime

```
session = ort.InferenceSession("/tmp/test_model.onnx")

print("✓ Model loaded in ONNX Runtime")
```

4. Compare outputs

```
with torch.no_grad():
    pytorch_output = model(dummy_input).numpy()
    ort_input = {'input': dummy_input.numpy()}
    ort_output = session.run(None, ort_input)[0]

# Check outputs match
diff = np.abs(pytorch_output - ort_output).max()

print(f"✓ Max output difference: {diff:.8f} (should be < 1e-5)")
```

5. Benchmark

```
input_np = np.random.randn(1, 784).astype(np.float32)
input_torch = torch.from_numpy(input_np)

# Warm up
for _ in range(100):

    with torch.no_grad():
        model(input_torch)
        session.run(None, {'input': input_np})
    # Measure PyTorch
    start = time.time()
```



```
for _ in range(1000):
    with torch.no_grad():
        model(input_torch)
    pytorch_time = (time.time() - start) / 1000
    # Measure ONNX Runtime
    start = time.time()

for _ in range(1000):
    session.run(None, {'input': input_np})
    ort_time = (time.time() - start) / 1000

print(f"✓ PyTorch: {pytorch_time*1000:.3f} ms/inference")

print(f"✓ ONNX RT: {ort_time*1000:.3f} ms/inference")

print(f"✓ Speedup: {pytorch_time/ort_time:.2f}x")

print("\n☑ ALL CHECKS PASSED — Environment ready!")

EOF
```

SECTION 2: macOS

PREREQUISITES

- macOS 12+ (Monterey or later)
- Python 3.9+ (via Homebrew: brew install python@3.11)
- pip installed

STEP 1: Create virtual environment

```
python3 -m venv ~/opt-env
source ~/opt-env/bin/activate
```

STEP 2: Install ONNX Runtime (CPU)

```
pip install onnx onnxruntime numpy torch torchvision
```

Verify:

```
python -c "import onnxruntime as ort; print('ONNX Runtime:', ort.__version__)"
```

Note: TensorRT is NOT available on macOS (NVIDIA GPU only).



Note: OpenVINO has limited macOS support — Intel CPUs only (not Apple Silicon).

For Apple Silicon (M1/M2/M3):

- ONNX Runtime works on ARM64 macOS
- Use CoreML execution provider for Apple-specific acceleration:

```
pip install onnxruntime (includes CoreML EP)
```

STEP 3: Install ONNX tools

```
pip install onnx onnxconverter-common
```

STEP 4: (Optional) Install OpenVINO — Intel Mac only

```
pip install openvino openvino-dev
```

```
python -c "import openvino as ov; print('OpenVINO:', ov.__version__)"
```

STEP 5: Run verification test (same script as WSL Section 1, Step 6)

SECTION 3: LINUX (Ubuntu/Debian)

PREREQUISITES

- Ubuntu 20.04+ or Debian 11+
- Python 3.9+
- pip installed

(Optional) NVIDIA GPU with CUDA for TensorRT

STEP 1: Create virtual environment

```
python3 -m venv ~/opt-env
```

```
source ~/opt-env/bin/activate
```

STEP 2: Install ONNX Runtime

```
# CPU-only (works for everyone)
```

```
pip install onnx onnxruntime numpy onnxscript
```

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

```
# GPU (requires CUDA)
```

```
pip install onnxruntime-gpu
```

Verify:

```
python -c "import onnxruntime as ort; print('ONNX Runtime:', ort.__version__); print('Device:',  
ort.get_device())"
```

STEP 3: (Optional) Install TensorRT

```
# Verify CUDA
nvidia-smi && nvcc --version
# Install via pip
pip install tensorrt

# Or use NVIDIA Docker (most reliable):
docker pull nvcr.io/nvidia/tensorrt:24.01-py3
docker run --gpus all -it nvcr.io/nvidia/tensorrt:24.01-py3
```

Verify:

```
python -c "import tensorrt as trt; print('TensorRT:', trt.__version__)"
```

STEP 4: (Optional) Install OpenVINO

```
pip install openvino openvino-dev

Verify:
python -c "import openvino as ov; print('OpenVINO:', ov.__version__)"
```

STEP 5: Run verification test (same script as WSL Section 1, Step 6)

SECTION 4: DOCKER APPROACH (Alternative — All Platforms)

If native installation is problematic, use pre-configured Docker containers:

ONNX Runtime GPU:

```
docker pull mcr.microsoft.com/onnxruntime/server:latest
TensorRT (includes ONNX RT + TensorRT):
docker pull nvcr.io/nvidia/tensorrt:24.01-py3
docker run --gpus all -it nvcr.io/nvidia/tensorrt:24.01-py3
```

OpenVINO:

```
docker pull openvino/ubuntu22_dev:latest
docker run -it openvino/ubuntu22_dev:latest
```

Advantage: pre-configured, no dependency conflicts.

Disadvantage: requires Docker, larger disk footprint.

SECTION 5: TROUBLESHOOTING

PROBLEM: "ModuleNotFoundError: No module named 'onnxruntime'"

CAUSE: Package not installed or wrong Python environment

FIX:

- Verify virtual environment is active: which python
- Reinstall: pip install onnxruntime

Check pip is for correct Python: pip --version

PROBLEM: onnxruntime-gpu installed but ort.get_device() returns "CPU"

CAUSE: CUDA version mismatch

FIX:

- Check CUDA version: nvcc --version
- Check cuDNN: apt list --installed | grep cudnn

Install matching onnxruntime-gpu version:

pip install onnxruntime-gpu==1.17.0 (for CUDA 12.x)

pip install onnxruntime-gpu==1.16.0 (for CUDA 11.x)

PROBLEM: torch.onnx.export fails with "Exporting operator X not supported"

CAUSE: Model uses operations not in ONNX spec

FIX:

- Increase opset version: torch.onnx.export(..., opset_version=17)
- Simplify the forward() method (avoid Python control flow)
- Use torch.jit.script for complex models before export

PROBLEM: ONNX Runtime output differs significantly from PyTorch

CAUSE: Usually floating point precision or dynamic shapes

FIX:

- Use fixed input shapes during export
- Set torch model to eval() mode before export
- Increase opset version

Check for random operations (dropout should be off in eval mode)

PROBLEM: TensorRT "pip install tensorrt" fails

CAUSE: System dependencies missing or CUDA version mismatch

FIX:

- Use NVIDIA Docker container (most reliable)
- Or install from NVIDIA's .tar archive (not pip)
- Ensure matching CUDA toolkit and cuDNN versions

PROBLEM: OpenVINO model conversion fails

CAUSE: Unsupported operations or model format issues

FIX:

- Convert via ONNX first: PyTorch → ONNX → OpenVINO
- Use latest OpenVINO version (broader op support)

Check OpenVINO supported operations list

PROBLEM: "CUDA out of memory" during ONNX Runtime GPU inference

CAUSE: Model too large for GPU memory

FIX:

- Reduce batch size
- Use FP16: session options enable mixed precision
- Free GPU memory: torch.cuda.empty_cache()

Monitor with: nvidia-smi

SECTION 6: VERIFICATION CHECKLIST

Run these and confirm expected results:

[] python -c "import onnxruntime; print(onnxruntime.__version__)"

→ Version displayed without error

[] python -c "import onnx; print(onnx.__version__)"

→ Version displayed without error

[] python -c "import torch; print(torch.__version__)"

→ Version displayed without error

[] Export test model to ONNX (script from Step 6)

→ "Model exported to ONNX" printed

[] Load model in ONNX Runtime

→ "Model loaded in ONNX Runtime" printed

[] Compare PyTorch vs ONNX Runtime outputs

→ Max difference < 1e-5

[] Benchmark PyTorch vs ONNX Runtime

→ ONNX Runtime faster (speedup > 1.0x)



[] (Optional) TensorRT import

→ Version printed if NVIDIA GPU available

[] (Optional) OpenVINO import

→ Version printed if Intel CPU

ALL CHECKS PASSED = Environment ready for Week 6 exercises